

СТАРТ

ДЕНЬ 5

Как создают программы в команде

Понять жизненный цикл изменения в команде: задача → ветка → коммит → review → merge → релиз.

DAY 05

SOURCE CODE

repository

branch

commit

bug

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

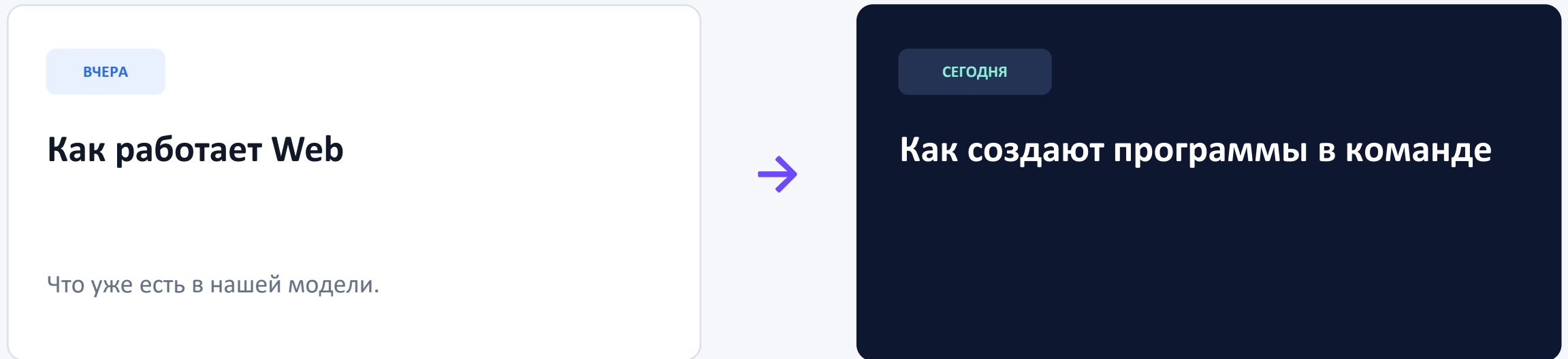
Конкретные действия вместо списка терминов.

- 01 Понять жизненный цикл изменения в команде: задача → ветка → коммит → review → merge → релиз.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.



Вчера: Как работает Web. Сегодня мы добавляем следующий слой: как создают программы в команде.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Даже идеальный код становится проблемой, если его нельзя безопасно проверить, обсудить и доставить в продукт.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Командная разработка похожа на совместное редактирование документа: каждый меняет свою копию, сохраняет понятную историю и только потом объединяет правки.

ПЕРЕВОД В IT

source code

текст программы, написанный разработчиком

repository

хранилище проекта и истории его изменений

branch

изолированная линия изменений в репозитории

Слой 1: source code • repository • branch

Сначала смысл, затем профессиональный термин.

**source code**

текст программы, написанный разработчиком

**repository**

хранилище проекта и истории его изменений

**branch**

изолированная линия изменений в репозитории

Слой 2: commit • bug • release

Сначала смысл, затем профессиональный термин.

**commit**

зафиксированный снимок осмысленного изменения

**bug**

ошибка, из-за которой программа работает не так, как ожидается

**release**

подготовленная версия продукта, выпущенная пользователям

Слой 3: version control • pull request

Сначала смысл, затем профессиональный термин.

version control

система хранения истории изменений проекта

pull request

предложение проверить и объединить изменения

Не заучивай формулировку. Придумай свой пример использования каждого термина.

ТЕОРИЯ

Слой 4: issue • changelog

Сначала смысл, затем профессиональный термин.

**issue**

описанная задача или проблема проекта

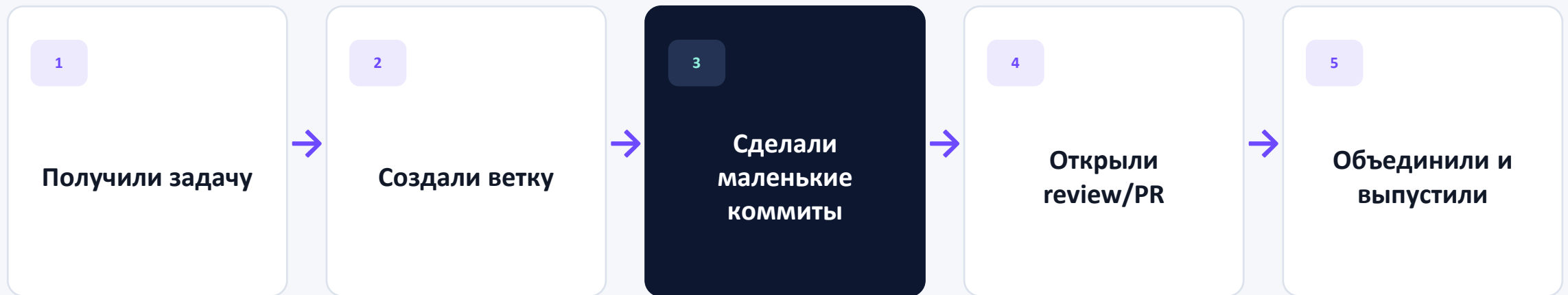
**changelog**

список изменений между версиями продукта

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day05_step1

```
task = "исправить вход"  
branch = "fix/login"  
commit = "fix empty password check"  
  
print(task)  
print(branch)  
print(commit)
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
task = "исправить вход"
```

Сохраняем значение или результат вычисления под понятным именем.

```
branch = "fix/login"
```

Сохраняем значение или результат вычисления под понятным именем.

```
commit = "fix empty password check"
```

Сохраняем значение или результат вычисления под понятным именем.

```
print(task)
```

Показываем результат, чтобы увидеть состояние программы.

```
print(branch)
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
task = "исправить вход"
branch = "fix/login"
commit = "fix empty password check"

print(task)
print(branch)
print(commit)
```

РЕЗУЛЬТАТ

```
исправить вход
fix/login
fix empty password check
```

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.

```
author = "Anna"  
reviewer = "Max"  
print(author, "пишет код")  
print(reviewer, "проверяет изменения")
```

changed



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

Anna пишет код
Max проверяет изменения

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



```
commit = "fix"  
print(commit)
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Код выполнится, но сообщение слишком общее. Через месяц никто не поймёт, что именно исправлялось. Это логическая/командная ошибка, а не `SyntaxError`.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.

```
commit = "fix empty password validation"  
print(commit)
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Провести маленький баг «пустое имя принимается формой» через рабочий процесс команды.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Описать ожидаемое поведение.**
- 2 Создать отдельную ветку.
- 3 Исправить проверку.
- 4 Сохранить понятный коммит.
- 5 Отправить на review и после проверки объединить.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
task = "reject empty name"  
branch = "fix/empty-name"  
commit = "validate empty name"  
status = "ready for review"
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.

```
task = "reject empty name"  
branch = "fix/empty-name"  
commit = "validate empty name"  
status = "ready for review"  
  
print(task, "->", branch, "->", commit, "->", status)
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

reject empty name -> fix/empty-name -> validate empty
name -> ready for review

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Расставь по порядку: merge, task, review, branch, commit.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Сначала понимаем работу, затем изолируем изменения, сохраняем историю, проверяем и только потом объединяем.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
steps = ["task", "branch", "commit", "review", "merge"]  
for number, step in enumerate(steps, start=1):  
    print(number, step)
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

1 task
2 branch
3 commit
4 review
5 merge

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Собери мини-план изменения кнопки «Купить» от задачи до релиза.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



```
flow = ["task", "branch", "code", "commit", "review", "merge", "release"]  
print(" -> ".join(flow))
```

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

task -> branch -> code -> commit -> review -> merge -> release

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Коммитить огромные несвязанные изменения.

02

Писать бессмысленные сообщения коммитов.

03

Работать всем прямо в одной ветке.

04

Считать code review поиском виноватого.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «source code» и какую проблему это понятие помогает описать?

2

Чем «source code» отличается от «commit»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **source code — текст программы, написанный разработчиком.**

2 source code: текст программы, написанный разработчиком. commit: зафиксированный снимок осмысленного изменения.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3

исправить вход
fix/login
fix empty password check

4

Код выполнится, но сообщение слишком общее. Через месяц никто не поймёт, что именно исправлялось. Это логическая/командная ошибка, а не `SyntaxError`.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Понять жизненный цикл изменения в команде: задача → ветка → коммит → review → merge → релиз.
- ✓ Читать и объяснять пример темы «Как создают программы в команде».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Алгоритмы и структуры данных

Завтра: Алгоритмы и структуры данных. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.